

Tema-3.-Jerarquia-de-Memoria.pdf



Anónimo



Arquitectura de Computadores



2º Grado en Ingeniería Informática del Software



Escuela de Ingeniería Informática
Universidad de Oviedo

Máster

Online en Ciberseguridad

Nº1 en España según El Mundo



Hasta el 46%
de beca



Mejor Máster
según el
Ranking de
ELMUNDO

Para ser el mejor hay que aprender
de los mejores.

IMEF

Smart Education

Deloitte

Infórmate

Que no te escriban poemas de amor
cuando terminen la carrera ▶▶▶▶▶▶▶▶▶▶



WUOLAH

(a nosotros por suerte nos pasa)

No si antes decirte
Lo mucho que te voy a recordar

Pero me voy a graduar.
Mañana mi diploma y título he de
pagar

Llegó mi momento de despedirte
Tras años en los que has estado mi
lado.

Siempre me has ayudado
Cuando por exámenes me he
agobiado

Oh Wuolah wuolah
Tu que eres tan bonita

LA JERAQUIA DE MEMORIA

Existen tres tecnologías diferentes de memoria:

SRAM: memoria caché.

DRAM: memoria principal.

Almacenamiento secundario: memoria virtual.

1. Introducción

Como la CPU es más rápida que la memoria, las instrucciones que acceden a ellas pueden causar detenciones.

La memoria ha evolucionado, principalmente, en densidad (capacidad) pero no en velocidad. Por ello, el coste de espera de la CPU es cada vez mayor y hace que el rendimiento efectivo baje.

Para solucionar los problemas ocasionados por el tamaño y la velocidad:

RAM estática (SRAM): en el procesador

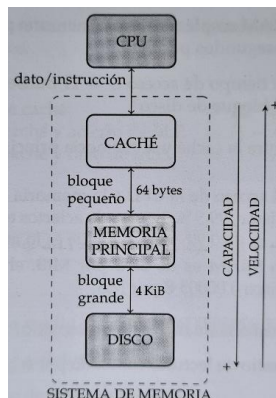
RAM dinámica (DRAM): requiere de refresco.

Almacenamiento secundario (HDD y SSD)

Las memorias SRAM son las más rápidas y las de menor capacidad. Producen un alto consumo de energía.

En cambio, el almacenamiento magnético es mucho más lento, pero de mayor capacidad y no consume energía.

2. Concepto de jerarquía



El objetivo para construir una memoria ideal sería que fuese de alta velocidad, capacidad y bajo coste. Para ello, se necesitan combinar memorias rápidas y pequeñas (datos con mayor probabilidad de acceso) con lentas y grandes (datos con menor probabilidad de acceso). Para saber que datos son los que tienen mayor probabilidad de acceso se utiliza el principio de localidad (observado en la ejecución de programas).

Principio de localidad espacial: si en un momento se accede a una dirección de memoria, es probable que poco después se acceda a una dirección cercana a la primera.

Principio de localidad temporal: se accede a una dirección de memoria y es probable que se vuelva a acceder a esa misma dirección (bucles).

Una CPU MIPS64 puede acceder a doubleWord, Word, halfWord y Byte. En el caso de emisión múltiple, el ancho del canal debe de permitir leer simultáneamente tantas instrucciones como determina el ancho de emisión.

Pre-fetching: lectura de varias instrucciones al mismo tiempo (principio de localidad espacial).

Cuando la CPU desea leer el contenido de una dirección de memoria, accede a la memoria caché. Si lo que se busca se encuentra en ella, se produce un acierto de caché y se encontró de la forma más rápido posible. Por el contrario, se produciría un fallo de caché y se transfiere un conjunto de posiciones de memoria principal (bloque) a la caché. Cada vez que se produce un fallo en los sistemas de memoria, se pasa el bloque de código contiguo en memoria principal a la caché. Y, si el dato no está en memoria principal (fallo de memoria principal), se pasa un bloque del disco a la caché.

$$t = Ac * tc + (1 - Ac) * [Ap * tp * Bp + (1 - Ap) * td]$$

Ac : tasa aciertos caché

tc : tiempo de acceso a caché

Ap : tasa aciertos m. principal

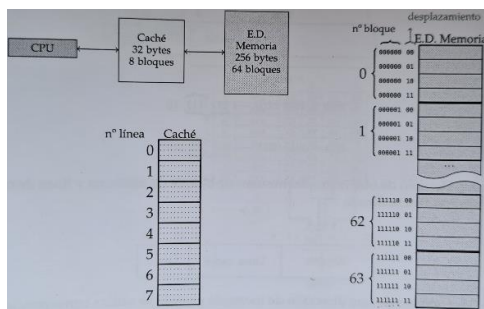
$tp * Bp$: tiempo para llevar un bloque de m. principal a caché

td : tiempo para llevar un bloque de disco a m. principal

3. La memoria caché

3.1. Conceptos preliminares

La memoria caché se implementa dentro del chip del procesador (SRAM). Está organizada en línea que contienen bloques de Bytes. Cuando la CPU intenta acceder a una dirección de memoria que no está cacheada, se produce un fallo de caché, se suspende el acceso a la misma y se copia en ella el bloque que contiene la dirección de memoria solicitada (se reanuda el acceso).



Los fallos de caché implican un incremento en el CPI (ciclos por instrucción). Una memoria caché de 32B organizada en líneas de 4B puede almacenar 8 bloques.

La dirección de memoria emitida por la CPU se divide en dos campos: nº de bloque y desplazamiento dentro del bloque. El bloque de memoria se obtiene teniendo en cuenta que los bits menos significativos de la dirección definen el desplazamiento de Bytes del bloque y los demás bits

proporcionan el nº de bloque de memoria (6b).

La memoria caché lleva asociado un controlador que gestiona lecturas y escrituras y comprueba si una dirección esta cacheada.

Características de funcionamiento:

Estrategia de correspondencia: establece a que línea caché se puede copiar cada bloque de memoria.

Estrategia de reemplazo: establece a que línea de caché debe de reemplazarse para albergar un bloque de memoria cuando se produce un fallo de caché.

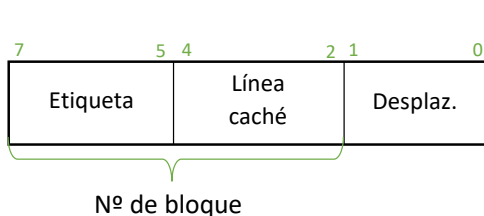
Estrategia de escritura: relación entre las escrituras en memoria caché y principal. Garantiza la coherencia entre la información de ambos niveles sin afectar al rendimiento.

3.2. Estrategias de correspondencia

* Correspondencia directa

Cada bloque de memoria se asigna siempre a la misma línea de memoria caché.

$$\text{Línea caché} = (\text{Bloque Mem}) \% (N^\circ \text{ de líneas cache})$$



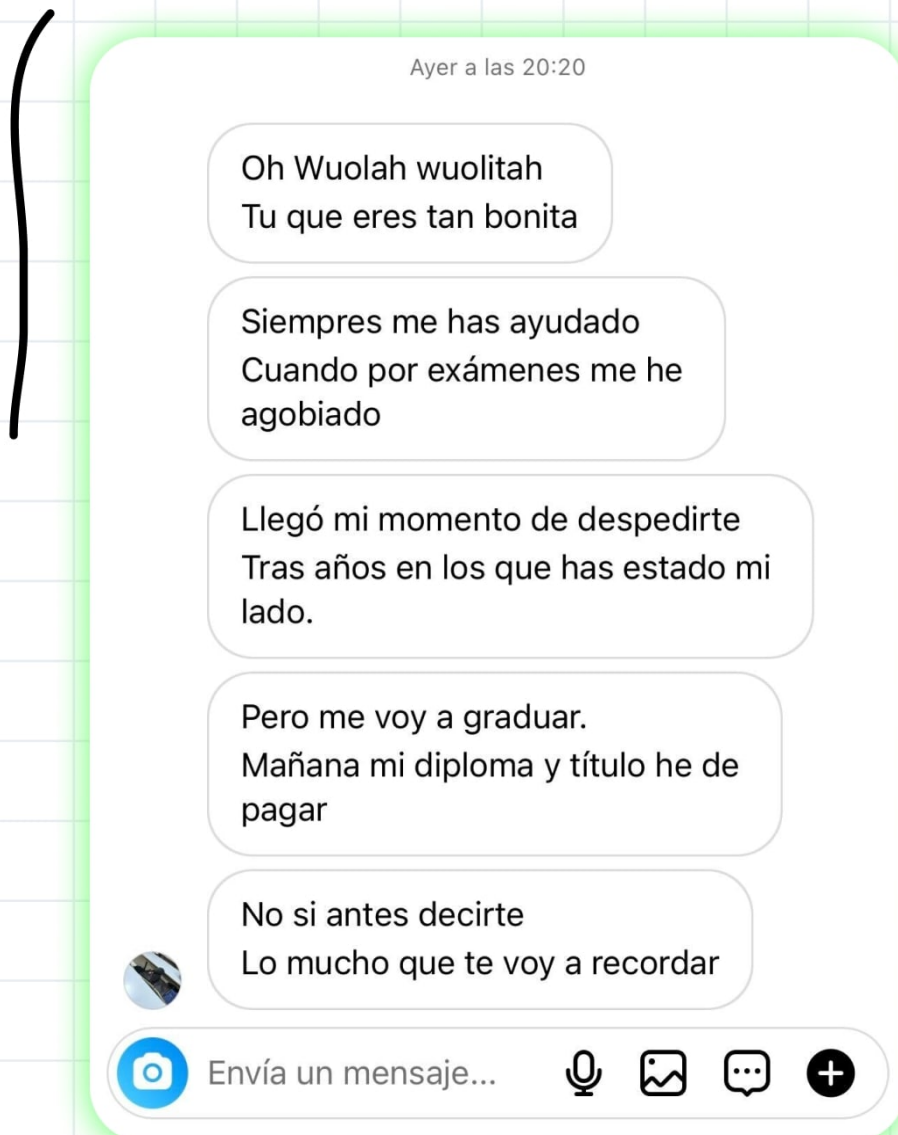
c

La línea de memoria caché se obtiene tomando los bits menos significativos del nº que representa al bloque de memoria

Cuando se copia un bloque de memoria a la cache se escribe además un conjunto de bits que identifican de forma inequívoca al bloque (etiqueta). Cada línea de caché tiene un bit de validez que indica si el bloque almacenado en la línea es válido.

Durante la inicialización del computador todas las líneas son inválidas.

**Que no te escriban poemas de amor
cuando terminen la carrera ▶▶▶▶▶▶▶▶**
(a nosotros por suerte nos pasa) 😊

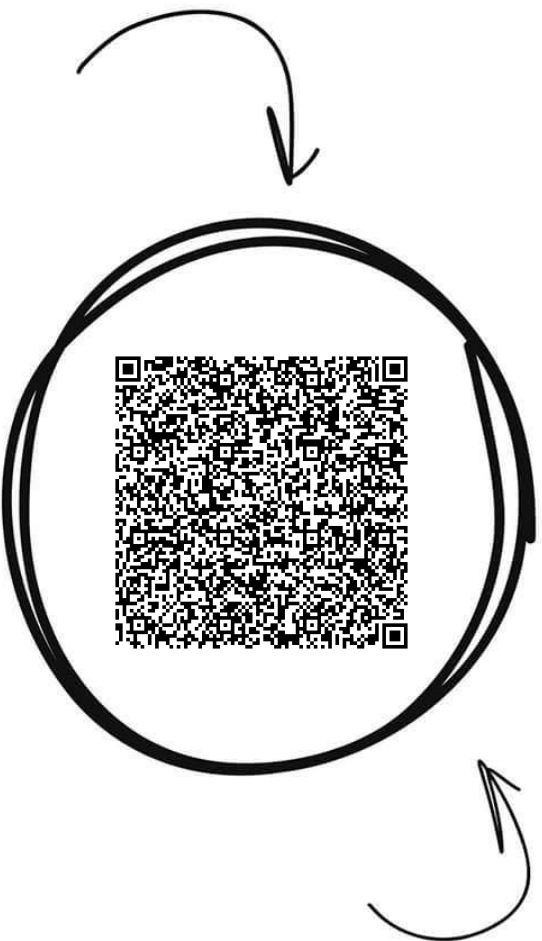


WUOLAH

Arquitectura de Computadores



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

WUOLAH

1

Imprime esta hoja

2

Recorta por la mitad

3

Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

4

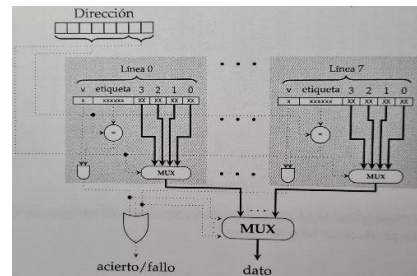
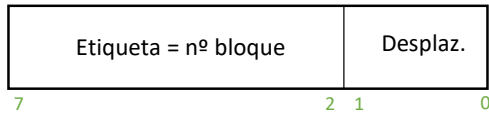
Llévate dinero por cada descarga de los documentos descargados a través de tu QR



Cuando se busca una dirección, se compara el campo etiqueta de ésta con la etiqueta de la línea. Si coinciden, el campo de desplazamiento indica el Byte de la línea a la que acceder.

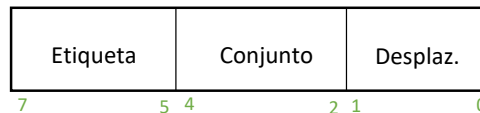
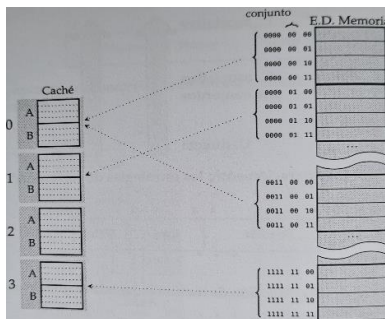
En el caso de que la CPU acceda con mucha frecuencia a dos bloques que tienen asignada la misma línea de caché, se producirán muchos fallos y provocará un bajo rendimiento.

* Correspondencia totalmente asociativa



Un bloque de memoria puede asignarse a cualquier línea de caché. La etiqueta de caché coincide con el nº de bloque de memoria. Esta opción presenta un rendimiento óptimo y alto coste de implementación para cachés grandes.

* Correspondencia asociativa por conjuntos



Utiliza un solo computador para toda la caché. Consiste en dividir la caché en conjuntos, cada uno de los cuales contiene un nº fijo de líneas. Puede cachearse en cualquiera de las líneas de un único conjunto.

$$N^{\circ} \text{ de líneas de un conjunto} = N^{\circ} \text{ de vías}$$

El nº de conjuntos de la memoria es 2^c , donde c es el número de bits empleados para representar al conjunto. Los bits menos significativos del bloque proporcionan el conjunto y, los restantes, la etiqueta.

Asociatividad máxima: Cuando el nº de líneas coincide con el nº de líneas de la caché.

3.3. Estrategia de reemplazo

Cuando se produce un fallo de caché se copia un bloque de código de la memoria principal a una línea de la memoria caché, reemplazando otro bloque. En el caso de la correspondencia directa solo hay un bloque que reemplazar.

Estrategias de reemplazo:

Least Recently Used (LRU): cada línea de cache almacena unos bits adicionales que indican cual es la que lleva más tiempo sin usarse.

Reemplazo aleatorio: se elige de forma aleatoria cualquiera de los bloques candidatos.

3.4. Estrategias de escritura

Write-through (escritura a través): toda operación de escritura se lleva a cabo simultáneamente en la caché y en el espacio de direcciones de la memoria.

Write-back (escritura diferida): los datos solo se escriben en la caché, sólo aparecen reflejados los cambios en la memoria cuando el bloque que lo contiene es reemplazado en la caché. Se ocasionan problemas de coherencia. (mayor rendimiento y complejidad hardware)

Estrategias de fallo de caché:

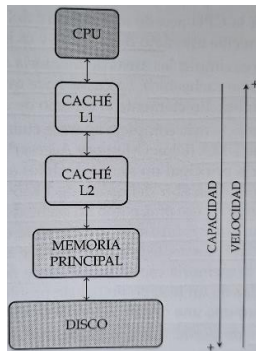
Write allocate: el bloque se copia en la caché antes de la escritura.

No write allocate: el bloque no se lleva a la caché, solo al espacio de direcciones de memoria.

* Se combinan: write-back con write allocate y write through con no write allocate.

Se asocia a cada línea un bit de “dirty” que se pone a 1 cada vez que la CPU escribe en la línea.

3.5. Organización de la caché



Existen dos factores determinantes en la organización de la caché:

- **Nº de niveles de caché:** lo habitual es disponer de 2 o 3 niveles (LLC ultimo nivel de caché). La L1 es la más cercana a la CPU y la de menor tamaño. En cambio, la LLC es la más alejada y esta compartida entre núcleos. Las cachés L2 y sucesivas se encargan de amortiguar los fallos de la L1. Cuantos más niveles de caché, mejor rendimiento.

- **Separación en cachés de datos y caches de código:** en la microarquitectura MIPS64 se incorporan memorias independientes para código y datos. Ésta, incluye dos caches: L1 de código y L1 de datos. Con esta distribución, podría suceder que la cache de datos tuviese líneas libres y la de código no. En el caso de una caché

unificada se reparten los bloques como sea necesario.

* Las primeras cachés eran unificadas para mejorar el rendimiento.

* Las L1 actuales se dividen por el grado de paralelismo. Permite escribir un dato mientras otra etapa puede leer código de instrucciones.

3.6. Coherencia de caché

La lectura realizada por la CPU sobre una dirección de memoria tiene que coincidir con el ultimo valor escrito por cualquier elemento del computador.

* **Coherencia de caché de un único nivel con una CPU**

Hay interfaces de periféricos ubicadas en el espacio de direcciones de la memoria. Por ello, si se realiza alguna acción con él, pasará automáticamente a la memoria. Si ese bloque de memoria no está cacheado, éste no contiene la información actualizada y si la CPU accede a la interfaz no dispondrá de una información correcta.

Las áreas de memoria asociadas a periféricos se marcan como no cacheables.

La interfaz de disco escribe en memoria principal un sector de datos solicitado cuando ya se dispone de él. Marcar dicha área como no cacheable ocasiona reducción del rendimiento de acceso a la CPU.

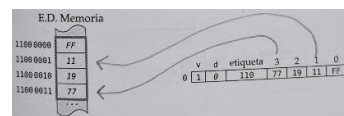
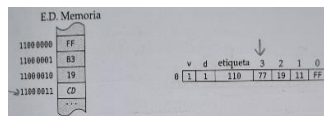
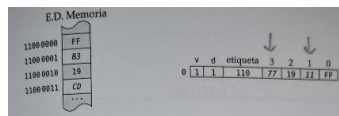
En periféricos DMA (Direct Memory Access) aparecen los siguientes problemas de coherencia:

- Un bloque cacheado ha sido modificado por un periférico. La lectura del bloque por parte de la CPU no está actualizada.

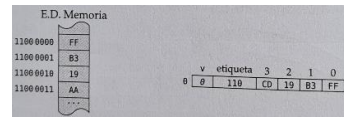
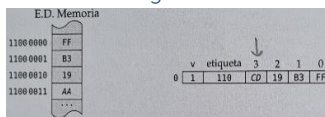
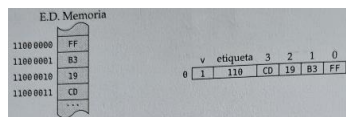
- La CPU modifica una línea de caché empleando “write-back”. Antes de reemplazar la caché, se lee de memoria principal y los datos no serán válidos. Se debe de emplear write-through.

Espionaje: en el caso de la escritura, la detecta e invalida la línea de caché. En el caso de la lectura, se detiene la lectura de memoria por parte del periférico, actualiza el bloque de memoria y permite la lectura.

Coherencia de caché en la lectura con “write-back”



Coherencia de caché en la escritura con “write-through”



4. Memoria virtual

Permite utilizar el disco como un nivel de jerarquía de memoria.

4.1. Introducción

La memoria virtual permite emplear la memoria principal como una caché de los dispositivos de almacenamiento secundario (discos duros). Éstas, proporcionan el nivel de jerarquía de mayor capacidad y menor velocidad.

Los programas que se ejecutan sobre las CPU que dan soporte a los SO multitarea que trabajan con direcciones virtuales traducidas a direcciones físicas por la MMU. Las direcciones físicas son las que realmente llegan a las líneas de dirección del sistema de memoria.

El espacio de direcciones virtuales representa todo el conjunto de direcciones virtuales que se pueden formar. Cada tarea tiene su propio espacio de direcciones virtuales (puede leer/escribir sin afectar a ninguna otra).

El espacio de direcciones físicas representa todo el conjunto de direcciones físicas que se pueden emitir (único para todo el sistema).

4.2. Memoria virtual paginada

Asignación de direcciones virtuales a direcciones físicas: se divide el espacio de direcciones virtuales en bloques que se traducen en bloques de direcciones físicas.

Los bloques (páginas) son de tamaño fijo. La dirección virtual se descompone en dos campos:



Desplazamiento: tamaño de la página, indica una dirección virtual dentro de la página. Si se suma a la dirección virtual de comienzo de página, se obtiene la dirección virtual.

Página: una página de la memoria virtual.

La dirección física se descompone en dos campos:

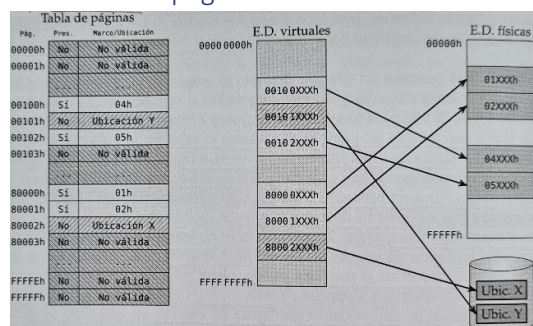


Desplazamiento: nº de bits dedicados al desplazamiento en la dirección virtual (el marco de página es el bloque de memoria física). Identifica una dirección física dentro de un marco de página. Si se suma a la dirección física de comienzo del marco, se obtiene la dirección física.

se obtiene la dirección física.

Marco: identifica un marco de página.

* La tabla de páginas



Se utiliza para convertir direcciones virtuales en direcciones físicas asociando páginas virtuales a marcos de páginas o áreas de disco.

Cada tarea tiene su propia tabla gestionada por el SO (se ubica en memoria física).

La tabla de páginas está formada por tantas entradas como páginas virtuales. En cada una de ellas:

Bit de presencia: indica si está en memoria (si esta inactivo quiere decir que está en el disco o no tiene

almacenamiento asociado y por lo tanto no puede ser accedida).

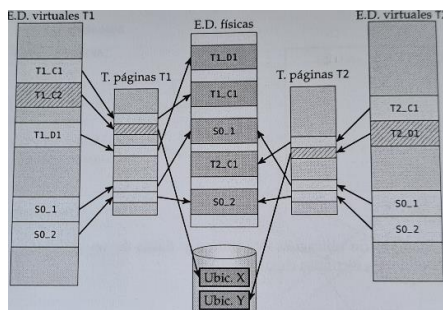
Marco de página/ubicación en disco: cuando el bit de presencia está activo indica el marco de página asociado a la pagina virtual. Si está inactivo, el SO le puede dar cualquier significado (proporciona área del disco asignado a la página virtual). Los SO disponen de áreas para implementar la paginación -> [archivo de](#)

paginación.

Bit de protección: establecen restricciones de lectura, escritura...

Bit de estado: se emplea durante el reemplazo de una página virtual en la memoria física y para cuantificar la frecuencia de acceso. Se emplean diferentes patrones de relleno para identificar diferentes tipos de páginas.

* Ubicación en memoria de las tareas y del SO



La codificación de instrucciones y datos (información adicional) las debe de gestionar el SO. Las herramientas de desarrollo ubican las instrucciones y datos sobre el espacio de direcciones virtuales asociando paginas virtuales.

Durante la carga de una tarea el SO asigna almacenamiento físico. Algunas páginas están en memoria física y otras en el archivo de paginación. Por ello, el SO debe de llevar una contabilidad de los huecos disponibles en memoria y en el archivo de paginación.

Además, modifica la tabla de páginas de la tarea reflejando la ubicación de las páginas virtuales de esta última (permite que las tareas se ejecuten correctamente sin importar de donde se carguen en memoria).

Algunos SO tienen mecanismos de seguridad que impiden que las direcciones virtuales sean fijadas en tiempo de ejecución y enlazado. Se generan direcciones reusables a las que se les asignan direcciones virtuales y líneas durante el proceso de carga.

El compilador y enlazador puede generar código a cualquier programa dentro de su espacio de direcciones virtuales.

El SO se ubica dentro del espacio de direcciones virtuales de todas las tareas. La transferencia de control al SO se lleva a cabo, si y solo si, la tarea no supone un cambio en el registro de tablas de páginas.

El marco de página asociado al SO es siempre el mismo.

* Protección de memoria

Independencia de los espacios de direcciones de tareas: el SO programa adecuadamente las tablas de páginas de las distintas tareas para que no haya solapamientos entre los marcos de páginas asociados.

Tipos de accesos a las páginas virtuales: campo que especifica si la página puede ser leída, leída y escrita, ejecutada, etc.

Nivel de privilegio: privilegio mínimo con el que debe de ejecutarse una instrucción para poder acceder a la pagina virtual. Está el nivel bajo (accedida por tareas y el SO) y el nivel alto (accedido por el SO).

Bit L/E = 1 -> lectura

Bit U/S = 1 -> usuario

Bit L/E = 0 -> lectura y escritura

Bit U/S = 0 -> supervisor

* Compartición de la memoria

El SO lo permite programando adecuadamente las tablas de páginas. Permite comunicar a gran velocidad dos tareas y evita duplicas en la memoria física. Todas las tareas comparten la memoria física asociada al SO. Con el uso de bibliotecas de enlace dinámico se evita la duplicación.

* Ampliación de capacidad de memoria

Podría considerarse que la memoria principal actúa como memoria caché del disco.

La CPU emite una dirección virtual y el campo nº de esta se emplea como índice dentro de a tabla de páginas de la tarea.

Si la página se corresponde con un marco, la MMU genera la dirección física correspondiente a la memoria virtual t accede al dato en memoria principal.

Si la tabla de páginas indica que está dentro del disco, se produce un fallo de página, se ejecuta el manejador y mueve la página entre el disco y la memoria principal. Esta técnica reduce el rendimiento del sistema.

Pasos seguidos por el manejador de excepciones:

1. Establece la causa del fallo: dirección sin almacenamiento asignado o en disco.
2. Si se trata de una dirección sin almacenamiento significa que no se encuentra ni en memoria física ni en el archivo de paginación. Se produce una violación de acceso a memoria.
3. Si se trata d una dirección almacenada en el disco el SO determina en que marco de página se va a cargar la página (utiliza el algoritmo de reemplazo). Se carga desde el disco la página solicitada y se actualiza su entrada en la tabla de páginas. El manejador retorna a la instrucción causante de la excepción.

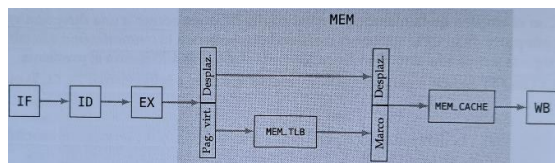
Working set: conjunto de marcos de página asignados a una tarea.

4.3. El TLB

Cada acceso a la memoria virtual necesita dos accesos a la memoria: uno a la tabla de páginas (accede a la memoria física para leer la entrada de la tabla) y otro a la dirección física asociada (accede a la dirección física resultado de traducir la dirección virtual por la entrada de la tabla). Este doble acceso a memoria reduce el rendimiento.

Una tarea accede a unas páginas virtuales cuyas entradas pueden almacenarse en una pequeña memoria caché totalmente asociativa. El acceso a una dirección virtual se reduce a un acceso al TLB.

Con cada cambio de contexto se camia el contenido de la tabla de páginas. El SO marcará como inválidas todas las entradas TLB (con cada cambio de contexto se produce un fallo de TLB).



Una posible solución sería añadir un conjunto de bits a cada una de las entradas del TLB que identifiquen la tarea a la pertenecen las entradas. Se marcarían ciertas entradas como globales (válidas para todas las tareas).

Un único TLB para todas las tareas -> capacidad mucho mayor.

Un acceso a memoria requiere de: acceso a caché -> acceso a TLB -> acceso a caché (2 accesos a caché, penaliza el rendimiento).

Mem_TLB y mem_cache: cada una un ciclo de reloj. La 1ª obtiene el marco de página a partir de la página virtual y la 2ª accede a caché con la dirección física obtenida.

En condiciones ideales, podría trabajar en paralelo y el tiempo de acceso a memoria sería de un ciclo.

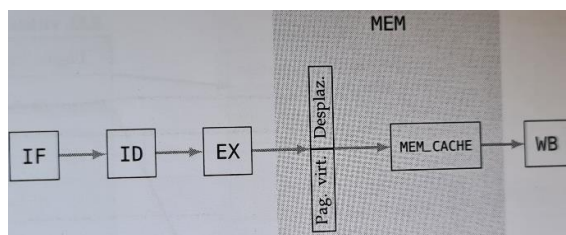
* Cachés virtuales

Emplean parcial/totalmente direcciones virtuales para su funcionamiento. Si la caché trabajase con direcciones virtuales, se podría acceder a ella sin esperar a que el TLB proporcionase el marco de página asociado a la página virtual.

Cachés indexadas y etiquetadas físicamente (P/P): tanto el conjunto caché como la etiqueta vienen de la dirección física (necesario acceder antes a TLB que a la caché).

* L2 y L3 trabajan con direcciones físicas.

Cachés indexadas y etiquetadas virtualmente (V/V): trabajan con direcciones virtuales. Para acceder a la caché no se necesita el TLB. Cuando la dirección virtual está cacheada, L1 sirve directamente el dato en un ciclo de reloj. Cuando se produce un fallo, accede al TLB y luego a L2. La caché L1 de V/V introduce varios problemas: invalida la caché con cada cambio de contexto, la caché debe de gestionar la protección de





(a nosotros por suerte nos pasa)

No si antes decirte
Lo mucho que te voy a recordar

Pero me voy a graduar.
Mañana mi diploma y título he de
pagar

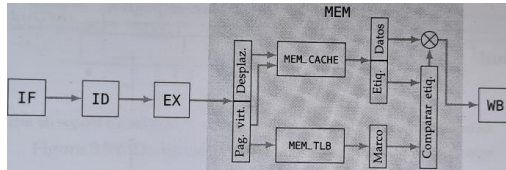
Llegó mi momento de despedirte
Tras años en los que has estado mi
lado.

Siempre me has ayudado
Cuando por exámenes me he
agobiado

Oh Wuolah wuolah
Tu que eres tan bonita

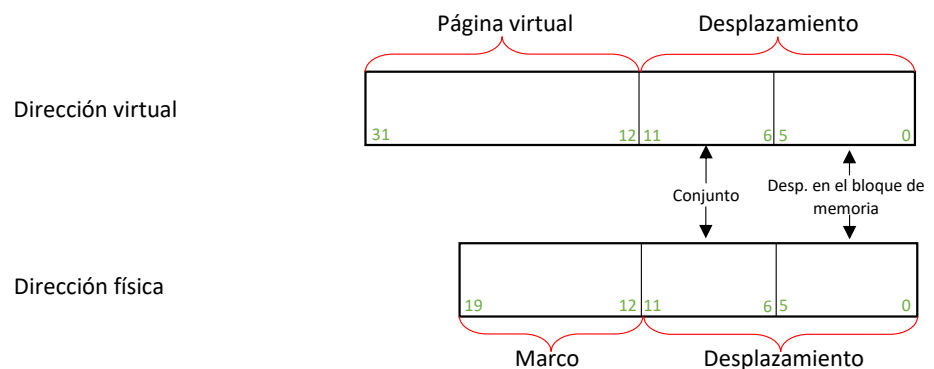
memoria y aparecen problemas de sinónimos (invalidar L1 con cada cambio de contexto, buscar sus sinónimos e invalidar las líneas que lo contengan).

Cachés indexadas virtualmente y etiquetas físicamente (V/P): La caché se indexa virtualmente, es decir, se emplean direcciones virtuales. El campo de etiqueta proviene de la dirección física. Para comprobar si la dirección virtual esta cacheada, debe compararse la etiqueta almacenada en cada vía del conjunto con la etiqueta obtenida a partir de la dirección física (se obtiene del TLB en paralelo). Requiere de un tiempo de acceso a memoria de 1 ciclo. Comparte el problema de sinónimos, pero con soluciones más simples.



Se debe de limitar el tamaño de la caché L1 y/o incrementar su asociatividad. Si el tamaño de página por nº de vías es mayor o igual al tamaño de caché, el conjunto en el que se cachea la dirección virtual coincide con la dirección física.

Si varias tareas comparten un bloque de memoria, se cacheará en el mismo conjunto sea cual sea la tarea.



5. Soporte de la virtualización

El espacio de direcciones físicas se muestra como múltiples espacios de direcciones virtuales (uno por tarea).

Con las máquinas virtuales se necesita un nuevo nivel de virtualización ya que hay SO que utilizan al mismo tiempo un único espacio de direcciones físicas que deben de compartir. Este es gestionado por el hipervisor que necesita un nivel de virtualización adicional ente la memoria virtual y física. Hay 3 espacios de direcciones:

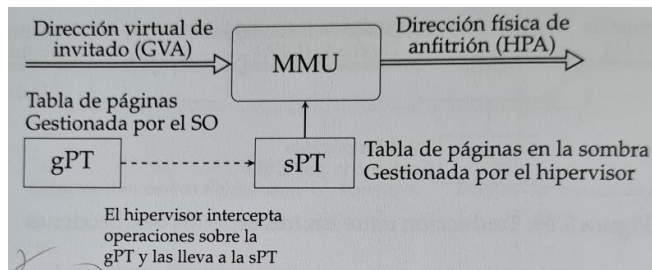
- Direcciones virtuales de invitado (las que percibe una tarea).
- Direcciones física de invitado (gestionadas por el SO en la memoria asignada a la máquina virtual).
- Direcciones físicas de anfitrión (reales del sistema por el hipervisor).

Para traducir la dirección virtual de una tarea a una dirección física se necesitan dos traducciones.

Dirección virtual de invitado (GVA) a dirección física de invitado (GPA): realizada por la MMU empleando la tabla de páginas gestionada por el SO. Tabla de páginas de invitado.

Dirección física de invitado (GPA) a dirección física de anfitrión (HPA): no hay ningún dispositivo en la CPU que la realice. Una 1ª solución es el empleo de las técnicas de las tablas de páginas en la sombra. Cada tarea tiene asociada una tabla (tabla de páginas en la sombra) gestionada por el hipervisor que contiene la traducción directa entre la GVA y HPA. La MMU, por tanto, trabaja con la tabla de páginas en la sombra (sPT) y no con la tabla de páginas de invitado (gPT).

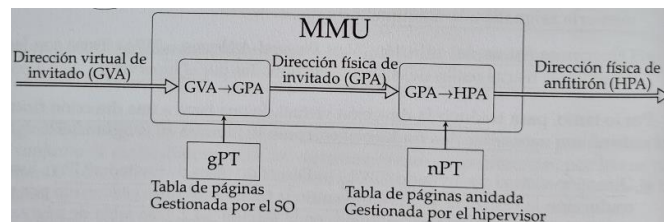
El hipervisor intercepta cualquier escritura del SO sobre gPT marcando todas las entradas como como de lectura. Cuando se intente modificar la entrada, saltará una excepción de fallo de pagina y el manejador modificará la entrada correspondiente.



El necesario que el hipervisor intercepte todas las instrucciones de gestión del TLB que ejecutan los SO. Un problema de las tablas de páginas en la sombra es el elevado nº de cambios de contexto entre el SO y el hipervisor, penalizando el rendimiento. Para evitarlo, se utiliza la traducción de 2º nivel (paginación anidada). Cuando una tarea accede a una

dirección virtual la MMU la traduce en una dirección física de invitado con la tabla gPT (por el sistema operativo anfitrión o hipervisor). El SO anfitrión puede escribir en la tabla y gestionar la TLB.

La MMU en una segunda fase traduce la dirección resultante en una dirección física del sistema empleando la tabla de páginas anidadas (bajo el control del hipervisor).



El TLB realiza una traducción directa de GVA a HPA durante la ejecución del SO y sus tareas.

Con la paginación anidada (nTP) no siempre mejora el rendimiento. El coste de fallo del TLB es mayor con paginación anidada.